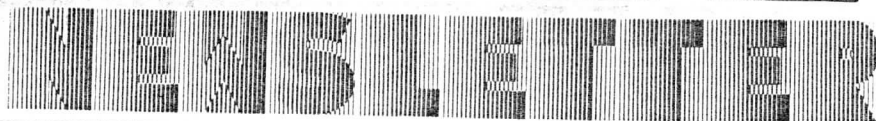
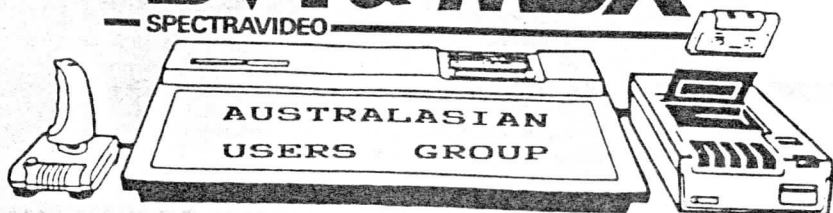


SVI & MSX

SPECTRAVIDEO



REGISTERED BY AUSTRALIA POST PUBLICATION No. TBH 0917 CATEGORY "B"

ISSUE NO.

3 - 4

ANNUAL SUBSCRIPTION

AUSTRALIA \$20.00
OVERSEAS \$25.00
OVERSEAS AIRMAIL ... \$30.00

YEAR BOOK \$20.00

DATE

JAN - 1986

CONTENTS

INTRODUCTION	2
EXPLORING BASIC Pt-15	3
IPL (PROGRAM)	8
MASTER MIND (PROGRAM)	9
BANK SWITCHING (PROGRAM)	14
LIBRARY NOTES	15

NEWSLETTER CORRESPONDENCE

S.A.U.G.,
P.O. BOX 191,
LAUNCESTON SOUTH,
TASMANIA, 7249.

(003) 442493

LIBRARY CORRESPONDENCE

S.A.U.G. LIBRARY,
1 CONRAD AVENUE,
GEORGE TOWN,
TASMANIA, 7253.

(003) 822919

Exploring Basic Pt-15*by L.A. Dunning.*

This part deals with disk drives and how to make the most of them from basic. It does not deal with standard ways of using disk statements as these are already fully covered in users manuals and the like. There are a number of approaches which are not quite apparent upon reading such books.

SEQUENTIAL FILES

The enormous advantage that a disk drive has over a cassette drive, is not so much time, as ease of access. In both cases you put the medium inside the drive and read/write from it, however with disk drives you don't need to press any keys, rewind any tape or similar physical tasks, just sit back and listen to the head eat through the disk. Any sector of a formatted disk can be accessed to be read or written to.

Now let's consider the means via basic of doing this. Firstly, we have the statements that mimic a cassette drive. These are OPEN-FOR-AS, PRINT#, INPUT#, SAVE, LOAD and CLOSE. Now a file on disk is created in the same manner regardless of content, be it a binary/ASC save or data file. In either case, the file starts at the first sector on a track. It continues using sequential sectors until all sectors on the track are used, then a new and unallocated track is picked and the sequence continues until you either run out of room on the disk or the file ends. A linked list on the File Allocation Table (FAT) keeps track of which track follows which on the file. If a file uses less than 17 sectors on a track, the remaining sectors cannot normally be used, unless for that file.

PRINT# actually transfers the data from its source not to the disk but to a buffer. When the buffer is full or the file closed, the contents of the buffer are dumped to a sector on the disk, the next sector is chosen and pointers reset to the start of the buffer. This process continues with the buffer being filled with data and then dumped to disk, until the last dump is made.

Since PRINT# and INPUT# are designed to imitate the cassette functions, it means that the data read/written advances along the sectors in "one direction". That is, it cannot read/write over previous information without either opening another file or closing and reopening the existing file.

RANDOM FILES

Now we come to statements that do not imitate cassette functions. These are OPEN-AS, GET, PUT, DSKO# & DSKI. The main difference with a random file is that the order of reading/writing to the disk does not have to be in sequence. When the disk is read or written to, only that sector is affected, not any previous sectors.

The structure of a random file on the FAT is identical to that of a sequential file. Now each record in a random file has a sector number, given in the second argument on a GET/PUT statement. The first track of a random file contains the first 17 records of that file, the 2nd track contains records 18 to 34, the 3rd contains records 35 to 51 and so on. When you GET or PUT a higher record than previously used, the system checks to see if that record is contained in the tracks already allocated to the file. If it is, it merely alters the end of file marker on the FAT and reads/writes to the sector. If it is not,

extra tracks are added to the file until the record can be accommodated. The contents of these new tracks, with the exception of the accessed sector, are not altered because of this.

Since each number in a GET/PUT statement equals a sector, the highest number you can go to on a normal BASIC disk is 612 sectors. On a disk that has the system tracks deleted (see previous issue) you can go up to 663 records. In either case, you would have just the one random file + directory track on disk.

DSKI# & DSKO#

Both DSKI# and DSKO# statements function in a similar manner as GET and PUT however you do not need to open a file to use them. Whereas with GET and PUT, a single record number is required, DSKI# and DSKO# require the numbers of disk, track and sector to be accessed. Both these statements work by reading and writing between buffer #0 and the sector in question. Both allow you to assign string variables, either to be dumped to or read. In this case the variable is being read to or from the buffer. The buffer is 256 bytes long and a variable is at most 255 bytes long. The last byte is ignored or lost!

The solution is to FIELD some dummy variables to buffer #0. Something like:

```
FIELD #0,128 AS B$(0),128 AS B$(1)
```

When using DSKO# to write back to disk, dump the data to the buffer and then, when using the statement, don't include a string expression. This enables you to write a full 256 bytes to a sector.

Since you can use these statements to access any sector on a formatted disk, you don't need a directory track. This gives you an extra 17 sectors to access, even though you can't tell using FILES what is on the disk. There are two disadvantages to using DSKI# and DSKO# statements instead of GET and PUT. You need physical parameters instead of just a record number. This could be a bonus if you want to encode data by its location, but a disadvantage in that it has to be calculated first. You also have to use buffer #0. This buffer, unlike buffers #1 and above, is used and cleared by the system on a regular basis and so is unreliable. The solution here is to use a second buffer and then dump the contents to/from buffer #0 before each access.

BUFFERS

Each buffer occupies 256 bytes in memory. The starting location of each buffer can be found by using VARPTR(#n) where n is the number of the buffer. The first 9 bytes are used for housekeeping by the system. The remaining 256 bytes (not 128, as stated elsewhere) are the contents of the buffer proper. Let's examine the first 9 bytes in detail.

Byte 0 is used to indicate the file mode. The values are (in octal):

001	Sequential File - INPUT
002	- - - OUTPUT
010	- - - APPEND
004	Random File
020	Delete File
200	Binary Save - BASIC PROGRAM

The first four values of this list are fairly straightforward. A Binary save is one where the basic program is saved in tokenised form instead

of ASC form. What does Delete File mean? I don't know. Perhaps it is used when truncating or deleting files but such a pointer would hardly seem necessary. Perhaps someone else would like to enlighten me on this. I'll certainly pass on the information.

Byte 1 points to the first entry for the file on the FAT. This is also the first track used for the file.

Byte 2 indicates the last track accessed by the buffer. This is not necessarily the last track used by the file. It also points to the current entry used on the FAT.

Byte 3 indicates the last sector accessed on the track indicated by byte 2.

Byte 4 indicates which disk is used by the buffer.

Byte 5 indicates the size of the buffer. This is normally set 0 which indicates a buffer size of 256. A shorter size might be given at the close of a sequential file (which is given by a control-Z on the disk sector).

Byte 6 gives the current PRINT position in the buffer. This is the location that PRINT# will start printing at or INPUT# read to.

Byte 7 is a flag indicating the following (in octal):

002	Buffer is empty
004	PUT has been done and no GET done yet
010	Buffer has been altered by PRINT#
020	File is write protected - "P"
100	File is set to read after write - "R"

The buffer empty flag is needed to distinguish between being full and empty, which are both represented by 0 in byte 6. The PUT flag is there because doing a PUT before either PRINT#/INPUT# are supposed to give errors unless a GET is done (see later).

Byte 8 indicates the number of bytes in the buffer/file since the last TAB or unsuppressed PRINT#. An unsuppressed PRINT# is one without a ";" behind it. This produces an extra carriage return (ODH) and linefeed (OAH) in the buffer. A TAB produces a 09H character in the buffer. These are dumped to/from disk as is.

MANIPULATING BUFFERS

According to certain manuals, you can open the same file under different names and then manipulate that file in different ways. This is almost true. If you try to open two files with the same name you will get an error. If you try to open two files with the same buffer you will also get an error. There are two ways around this.

Firstly, you could alter the Directory Table (DIR, see last issue) so that a file had two entries. Both would point to the same entry on the FAT. You could then open two separate buffers on the same material, using INPUT for one and OUTPUT or APPEND for the other. The problem here, is that the results would be unpredictable. The physical records of the file in question would be dependent upon the last buffer to be closed. You could lose the file entirely by having bad entries on the FAT.

The second way is to only open a random file buffer for the file. Instead of OPEN"filename"FORactionAS#1, use OPEN"filename"AS#1. The reason is, that both PRINT# and INPUT# both work in SV random files. The GET statement also works for sequential files, although PUT does not. Whenever a GET or PUT is performed, bytes 2 & 3 in the buffer point to the last sector accessed and byte 6 (the buffer offset) is set to 0. PRINT#s and INPUT#s made after this start at the beginning of the buffer with the sector as set by the last GET/PUT.

The advantage here, is that you can do both an INPUT# and PRINT# in the same buffer.

You may also use GET to position the buffer in sequential files. The only problem is that GET will dump the sector to the buffer and destroy any contents therein. Also, performing a GET greater than the length of the file will produce an error. The solution is to use LOF(#n) to determine the length of file, where n is the buffer used. This gives the number of records in the file.

VARIABLE FILE LENGTHS

One thing lacking with SV random files is the convenience of having variable lengths or records. On other basics you can set your record length between 1 and 256 records. On the SV, it is set to 256. If you have a random file and a record length less than 256, then the excess bytes are wasted if you only use GET and PUT. Assuming you only used 129 bytes for each entry, then the remaining 127 bytes are blanked out and lost.

There are several ways around this. If the file was one that was only to be read, you could use a combination of GET and INPUT#() to read the data. Assume the record length to be "L" where L is less than 256 and the record number to be "N". Use PRINTUSING# to place the records on disk originally with the length of the control string being equal to L. To retrieve data, do:

```
GET#1,(N*L\256)
POKE VARPTR(#1)+6,(N*L)MOD256
D$=INPUT$(L,#1)
```

To record data, do the first two statements and then substitute

```
PRINTUSING#1,DX$;D$,nn...nn;
```

where DX\$ is the control string for PRINTUSING and D\$/nn are the variables printed. The first one will work for reading data, the second should work all the time however I do not guarantee it.

The second way is to subdivide the buffer into subfiles. If the record length is 128 records or less, you can fit 256\L records in a sector. Assuming C to be the number of whole records you can fit into a sector and the original statement to be something like:

```
FIELD#1,L1asD$,L2asE$,L3asF$
```

then you alter it to be:

```
FIELD#1,L1asD$(0),L2asE$(0),L3asF$(0),L1asD$(1),L2asE$(1),L3asF$(1)...
```

and so on until you reach D\$(C-1),E\$(C-1) & F\$(C-1). This might take more than one statement and you can use dummy variables to pad out any further FIELD statements.

Assuming N to be the record to be accessed you do the following statements:

```
GET#1,(N\C)
P=N MOD C: D$=D$(P):E$=E$(P):F$=F$(P)
```

here D\$, E\$ & F\$ are the variables used by the program. To alter the record, you must do the above and also:

```
LSETD$(P)=D$:LSETE$(P)=E$:LSETF$(P)=F$
PUT#1,(N\C)
```

The important thing is, that the whole sector must be read before altering any part of it otherwise information contained in the other records will be lost.

There is no reason why different parts of a buffer have to contain duplicate records. You could have two separate files sharing the same physical record, by using dummy strings in FIELD statements. This is a variation on the above technique. Assuming you have file A

that requires 100 bytes per record, file B that needs 80 bytes per record and file C that also requires 80 bytes then you could open a file called "DATA" and do the following.

When you require File A:

FIELD#1,100asA\$,156asD1\$

When you require File B:

FIELD#1,100asD1\$,80asA\$,86asD2\$

When you require File C:

FIELD#1,180asD1\$,80asA\$,6asD2\$

here A\$ is the variable used to read the file and D1\$ and D2\$ are dummy variables. Once again, you have to GET a record before PUTting it to ensure that the other files aren't destroyed. The difference between this and the previous technique is that it is designed to be used by separate programs.

All the above attempt to conserve space on the disk at the cost of extra handling, still you can't have everything, It's an old question of speed vs versatility.

IPL

This months listing is IPL. This is the program that I normally use with the IPL command when the disk is booted up. It produces a title for the disk, executes a FILES and sets colour and function keys. When all this is done, it NEWS itself. This uses one track on the disk and no memory. I normally save this as the first program on disk and call it "ipl" so it is immediately recognizable. An interesting variation is to call it " " and save it as an ASC file. This produces no name when FILES is done, only a size listing. No doubt you have your own version of such a program. I'm including it for those who are new to disks; modify it for your own needs.

NEXT MONTH

In next months part, I'll show how to handle possible and probable errors using disks and how to repair files.

IPL

by : L.A. Dunning

This Program may be entered using the 'INPUT' program from Newsletter 2 - 2 (NOV. 84.) or The Year Book.

```
B6      10 REM  IPL
JJ      20 REM  Save program to disk as "ipl"           then type:
NJ      30 REM  IPL "RUN"+CHR$(34)+"1:ipl"             and reboot computer
BD      40 MAXFILES=4:RS$=CHR$(27)+"p":RE$=CHR$(27)+"q":QU$=CHR$(34):CL$=CH
        R$(12):CR$=CHR$(13):CU$=CHR$(21)
CA      50 COLOR15,1,3:CLS:READA$:PRINTRS$:PRINTUSING"<< \" +SPACE$(23)+\" F
        REE ## >>";A$;DSKF(1):PRINTRE$:FILES
EK      60 DATA <---Name of Disk----->
IN      70 KEY 1,CU$+"FILES"
CO      80 KEY 2,"LOAD"+QU$+"1:"
FH      90 KEY 3,"SAVE"+QU$+"1:"
EB      100 KEY 4,CU$+"list"
HN      110 KEY 5,CU$+"run"+CR$
IG      120 KEY 6,CU$+"color15,1,5"+CR$
GN      130 KEY 7,"MERGE"+QU$+"1:"
AF      140 KEY 8,"KILL"+QU$+"1:"
ED      150 KEY 9,CU$+"list."+CR$
EG      160 KEY10,CL$+"run "+CR$
DG      170 NEW
END
```

BANK SWITCHING

by : T. Colverd

This Program may be entered using the 'INPUT' program from Newsletter 2 - 2 (NOV. 84.) or The Year Book.

```
ED      5 REM  FFF THIS PROGRAM WILL NOT WORK UNDER DISK BASIC 444
EF      6 REM  FFF      ONLY FOR SV-328 OR 318 WITH 64K RAM      444
BO      100 CLEAR 200,%HF480
AL      110 DEFUSR=%HF480:DEFINT I
BE      120 FOR I=%HF480 TO %HF4F9
DB      130 READ A$:POKEI,VAL("%H"+A$)
AI      140 NEXT I:I=USR(0):NEW
AL      150 DATA 22,5E,FE,E5,3E,C3,32,57,FF,21,B4,F4,22,5B,FF,ED
AH      160 DATA 73,5C,FE,F3,3E,0F,D3,88,DB,90,32,64,FE,E6,FD,D3
DE      170 DATA 8C,21,00,80,11,00,00,01,00,80,ED,80,3A,64,FE,D3
CK      180 DATA 8C,FB,E1,C9,FE,C9,C0,ED,73,5C,FE,DD,2A,03,FA,23
AI      190 DATA 22,5E,FE,F3,3E,0F,D3,88,DB,90,32,64,FE,E6,FD,D3
BF      200 DATA 8C,21,00,00,11,00,80,4E,EB,46,71,EB,70,23,13,7A
GM      210 DATA B3,20,F4,3A,64,FE,D3,8C,FB,ED,7B,5C,FE,CD,50,37
CC      220 DATA DD,22,03,FA,2A,5E,FE,7E,C1,C9
END
```

This program allows you to use the SVI SWITCH command without the need of a second Bank of 64K ram.

Please note that the program only works with the disk drives turned off.

Anyone with a fix for this please call us.